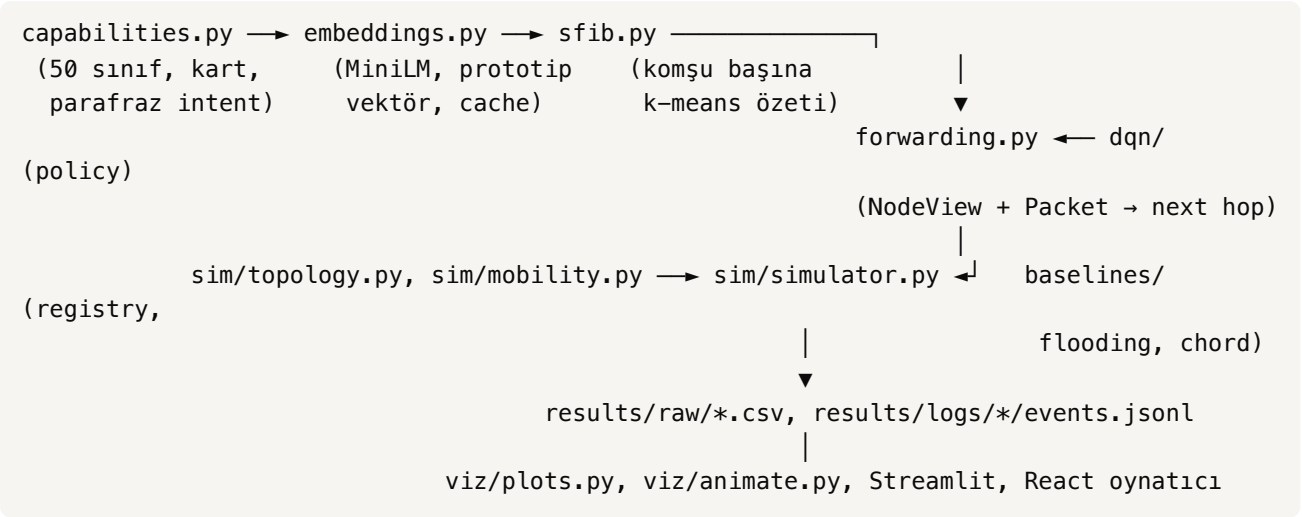
AgentRoute: Nasıl Yaptık

Bu belge her parçanın nasıl tasarlandığını, hangi kararların neden alındığını ve yolda karşılaşılan sorunları anlatır. Kim neden sorumluydu sorusunun cevabı `KIM_NE_YAPTI.md` dosyasında. Sunumdan önce üç kişinin de bu belgenin tamamını okuması gerekir.

1. Büyük resim



İki ilke bütün tasarımı belirledi:

1. **Forwarding simülatörden bağımsız.** `forwarding.py` sadece bir düğümün yerel durumunu (`NodeView`) ve paketi görür, bir komşu döndürür. Simülatör bu fonksiyonu her hop'ta çağırır. Aynı kod ileride bir UDP overlay'e taşınabilir.
2. **Simülasyon ve görselleştirme ayrı.** Simülatör her olayı JSONL'e yazar. Grafikler, animasyon, Streamlit ve React oynatıcı sadece bu dosyaları okur.

2. Adresleme ve S-FIB (Bekir Emre)

2.1 Dataset

- 50 yetenek sınıfı, altı grupta: görüntü, dil, ses, veri/analitik, mobilite-IoT-ağ ve diğer servisler.
- Her sınıfta:
  - **2 yetenek açıklaması.** İki farklı sağlayıcının aynı yeteneği farklı kelimelerle anlatmasını temsil ediyor. Örnek: "object detection in images" ve "locate and label every object in a photo with bounding boxes".
  - **3 örnek istek.** A2A agent kartlarındaki `examples` alanı gibi. Örnek: "box the vehicles in a traffic camera image".
  - **10 parafraz intent.** Kullanıcının gerçekten yazacağı türden günlük cümleler. Örnek: "find the cars in this photo".
- Intent'ler sınıf başına sabit seed ile 7 train / 3 test olarak bölünür (350 / 150). DQN ve τ sadece train ile ayarlanır, bütün deneyler test intent'leriyle yapılır.
- `tests/test_core.py` her intent'in hiçbir kart cümlesini içermediğini ve hiçbir kart cümlesinin intent'i içermediğini kontrol ediyor. Bu test bir gerçek çakışmayı yakaladı ("compose a follow-up note" örneği bir intent'in içinde geçiyordu) ve örnek değiştirildi.

2.2 Capability address = kartın prototip vektörü

İlk denemede her agent'ın adresi tek bir kısa açıklamanın embedding'iydi. Ölçtük:

Adres	Test top-1 doğruluk
Kısa açıklama ("object detection in images")	0.61–0.65
İki açıklama birleşik	0.61
Açıklama + 2 örnek istek, tek metin olarak	0.68–0.69
Açıklama + 2 örnek istek, embedding'lerin ortalaması (prototip)	**0.73–0.77**

Sonuç: MiniLM konu benzerliğini ölçüyor, "görev türü" benzerliğini değil. "who was the first president of Turkey" gibi bir istek "question answering over a knowledge base" ile zayıf eşleşiyor. Kartın cümlelerinin embedding'lerinin ortalaması (prototip, few-shot sınıflandırmadaki prototypical networks fikri) bu açığı kapatıyor. Adres hâlâ tek bir 384 boyutlu vektör, böylece CLAUDE.md'deki "adres = vektör + agent ID" tanımı korunuyor. Kod: `embeddings.card_vector()`.

2.3 Kabul eşiği τ

- Agent bir işi cos(intent, adres) ≥ τ ise kabul eder.
- τ, train intent'leri ile bütün kartlar arasındaki çiftlerde F1'i en yüksek yapan değer seçildi: aynı sınıf = pozitif, farklı sınıf = negatif.
- Sonuç τ = 0.405: train precision 0.64, recall 0.54. Testte kartlarla top-1 doğruluk 0.77, yanlış kabul oranı %0.6 (`results/tau.json`).
- Neden F1? Ağda bir isteğin yolunda onlarca agent var. Düşük bir τ (yüksek recall) yanlış sınıftaki agent'ların işi kapmasına yol açar. Precision'ı da hesaba katan F1 bu yüzden doğru ölçüt.

2.4 S-FIB

- Her router, her komşusu için `counts[komşu, kart, hop]` sayaç tablosu tutar (numpy dizisi): o komşunun arkasında, d hop uzakta, o kartla kaç agent var.
- **Duyuru kuralı (R-hop sınırlı mesafe-vektörü):**
  - u router'ı, v router'ına bağlı agent'ı n komşusu üzerinden öğrenir, eğer dist(u, v) ≤ R ve dist(n, v) = dist(u, v) − 1 ise (n, v'ye giden en kısa yollardan birinin üzerinde).
  - Bu, R hop sonra yayılmayı durduran bir distance-vector protokolüyle aynı sonucu verir.
  - Hesaplama: v'den R derinliğe kadar BFS. Sonuç her v için cache'lenir.
- **Özet:**
  - Komşunun arkasındaki kartlar k = 4 temsilciye indirilir (spherical k-means).
  - Başlangıç deterministik (en ağır vektör, sonra en uzak nokta), böylece aynı tablo hep aynı özeti verir.
  - Her temsilci, kümesindeki en kısa hop mesafesini de taşır.
  - Karşılaştırma için `mean` (tek ortalama vektör) ve `exact` (sıkıştırmasız) seçenekleri var.
  - Komşu skoru = intent ile temsilciler arasındaki en yüksek cosine.
- **Handover (artımlı güncelleme):**
  - Eski hücre withdraw (delta = −1), yeni hücre announce (+1) yapar.
  - Güncellemeler hop başına ~5 ms gecikmeyle mesafe sırasıyla yayılır. Yani kısa bir süre eski bilgiyle yönlendirme yapılır, bu gerçekçi.
  - Bir mesaj sadece bir komşunun duyurduğu özet (hangi kart, hangi en kısa mesafede) gerçekten değiştiyse sayılır. Aynı sınıftan başka agent'lar varsa çoğu handover çok az mesaj üretir.

- **Failure:** komşular çöken router'ı algılama süresinden (1 s) sonra fark eder. Tablolar canlı graf üzerinde yeniden kurulur, değişen her (komşu → router) özeti bir mesaj sayılır.
- **Periyodik refresh:** her 10 s'de, girdisi olan her yönlü link için bir duyuru mesajı ve her agent için bir kayıt mesajı sayılır (soft state).

2.5 Forwarding ve greedy

- **Paket:** intent vektörü, TTL (16), önceki hop, ziyaret edilen düğümler, oluşma zamanı. Chord için ayrıca istek metni.
- **Her router'da sırayla:**
  1. Bağlı agent'lardan cos ≥ τ olan en iyisi varsa ona teslim edilir.
  2. Yoksa policy bir sonraki hop'u seçer. Paket asla geldiği komşuya geri gönderilmez (çıkmaz sokak hariç).
- **Greedy:**
  - En yüksek S-FIB skoruna sahip komşuyu seçer. Eşitlikte daha az hop kazanır.
  - Ziyaret edilmemiş komşular tercih edilir.
  - Hiçbir komşuda girdi yoksa default route (en yakın core) kullanılır.
- **Özellikler:** `neighbor_features()` her aday komşu için 14 özellik çıkarır. DQN bunları kullanır:

Özellik	Anlamı
max_sim, mean_sim	intent ile komşunun temsilcileri arasındaki en yüksek / ortalama cosine
sim_gap	bu komşunun skoru eksi en iyi komşunun skoru
above_tau, has_entry	eşik üstü eşleşme var mı, komşuda hiç girdi var mı
hops_norm	en iyi temsilcinin hop mesafesi / (R + 1)
queue_occ	komşunun kuyruk doluluğu (hello mesajlarıyla, 100 ms'de bir)
link_quality, link_delay	1 − kayıp olasılığı, normalize gecikme
in_handover	komşu gNB'de son 2 s içinde handover oldu mu
ttl_ratio, visited	kalan TTL oranı, komşu daha önce ziyaret edildi mi
is_core, is_cloud	komşunun türü

3. Simülatör ve baseline'lar (Ömer)

3.1 Topoloji

- **gNB'ler:** hücre ızgarası, 500 m aralık, her gNB aynı zamanda bir edge router. Xn linkleri 4-komşuluk.
- **Core:** daha seyrek bir ızgara (~8 hücreye bir core), core'lar kendi aralarında ızgara.
- **Backhaul:** her gNB en yakın core'a bağlı. Hücrelerin yarısı (dama tahtası) ikinci en yakına da bağlı (failure deneyinde alternatif yol).
- **Cloud:** iki core'a 20 ms'lik linklerle bağlı.
- **Gecikmeler:** radyo 5 ms, Xn 2 ms, backhaul 5 ms, core 3 ms.
- **Kayıplar:** kablolu %0.05, radyo %0.2, kablolu linklerin %10'u "bozuk" (%3).
- **Ölçekleme:** hücre sayısı agent sayısıyla büyür (20 agent/hücre, en fazla 400 hücre). 10.000 agent'ta 400 hücre + 49 core + cloud = 450 router.

3.2 Router modeli

- **İstekler için:** her router bir SimPy `Resource` (servis 0.2 ms, kuyruk limiti 100). Registry sunucusundaki ve flooding'deki gerçek tıkanıklık buradan gelir.
- **Arka plan trafiği:**
  - Her paket, M/M/1 kuyruğundaki iş sayısı kadar arka plan paketi bekler. $P(N \geq n) = \rho^n$, bekleme Gamma(N, 1 ms). $N \geq 50$ ise paket düşer.
  - Normal router: $\rho = 0.2$.
  - **Hotspot router'lar (%15):** ortalama 5 s'de bir "yoğun" ($\rho = 0.95$) ve "normal" arasında geçer.
  - Yük çarpanı ρ'yu ölçekler (en fazla 0.97).
  - Bu yaklaşım arka plan paketlerini tek tek simüle etmeden gerçekçi bekleme ve kayıp üretir. 10.000 agent'lık koşular bu sayede saniyeler sürer.

3.3 Agent'lar ve mobilite

- **Dağılım:** %40 sabit edge agent, %50 mobil agent (random waypoint, hız ×[0.5, 1.5], 2 s duraklama), %10 cloud agent.
- **Yetenek arzı Zipf(1) dağılımlı:** birkaç yetenek çok yaygın, çoğu nadir. İstekler bütün sınıflardan eşit gelir. Bu kararı ölçerek aldık (bkz. 6.2).
- **Handover:**
  - Her saniye konumlar güncellenir.
  - UE, yeni gNB mevcut olandan %15 daha yakınsa geçer (histerezis, ping-pong'u önler).
  - Geçiş 50 ms sürer. Bu sırada agent'a giden paketler `handover` nedeniyle kaybolur.
- **Adil karşılaştırma:** topoloji, mobilite, trafik, ağ ve failure için ayrı random stream'ler var. Aynı seed'de her yöntem aynı agent'ları, aynı hareketi ve aynı istek dizisini görür (`test_all_methods_see_the_same_queries`).

3.4 Baseline'lar

- **Merkezi registry:**
  - Sunucu en merkezi core router'da, tek bir kuyrukta 1 ms servis süresiyle.
  - Kayıt, 10 s'de bir yenileme ve her handover sonrası güncelleme ağ üzerinden mesaj olarak gider ve sunucuyu meşgul eder.
  - Sorgu yolu: istek → sunucu → cevap → görev agent'ın bilinen hücresine.
  - Agent o arada taşındıysa `stale_location` olur.
  - Sunucu dolarsa `registry_queue`, router'ı çökerse her şey biter (tek hata noktası).
- **Flooding:**
  - Her router bir isteği bir kez işler ve bütün komşularına kopyalar (TTL 6).
  - Bağlı agent'larına tek bir yayın yapar. τ'yu geçen her agent cevap verir.
  - İstekçi 1 s bekler ve en benzer cevabı seçer.
- **Chord:**
  - Router'lar SHA-1 halkasında, finger table ile O(log N) lookup yapar. Her overlay hop alttaki ağda en kısa yol olarak gerçekten yürünür.
  - Anahtar = SHA-1(normalize edilmiş kart açıklaması).
  - Parafraz bir istek başka bir anahtara gider ve bulunamaz. Bu baseline'ı "anlamsal eşleşme neden gerekli" sorusunun referans noktası olarak kullanıyoruz.

3.5 Olay logu

CLAUDE.md'deki formatta JSONL (`query`, `hop`, `deliver`, `drop`, `handover`, `adv`, `withdraw`, `move`, `load`, `fail`) ve bir `topology.json`. Büyük deneylerde log kapalı, demo ve Streamlit'te açık.

4. Öğrenen next hop (Bekir Göktepe)

4.1 Model

- **Paylaşılan Q-ağı:** komşu sayısı değişken olduğu için ağ her komşuyu ayrı skorlar. Girdi = komşunun 14 özelliği, çıktı = tek bir Q-değeri. Action = en yüksek Q'lu komşu.
- **MLP:** 14 → 64 → 64 → 1. Bütün router'lar aynı ağırlıkları kullanır.
- **Batch'ler:** 32 komşuya kadar doldurulur ve bir maske ile işlenir.
- **GNN varyantı:**
  - Her komşu önce 64 boyuta gömülür.
  - Bütün komşuların ortalaması ve maksimumu bir bağlam vektörü olur. Bu, yerel yıldız grafiğinde tek tur mesaj geçişi.
  - Skor = MLP([komşu, bağlam]). Böylece "bu yol yoğun ama diğerleri de öyle" gibi göreli kararlar öğrenilebilir.

4.2 Reward

Olay	Reward
Her hop	−(0.05 + hop gecikmesi / 0.1 s)
Doğru agent'a teslim	+1
Düşme (TTL, kuyruk, link, yanlış agent, handover, ...)	−1

Ölçekler baştan normalize edildi. Bir hop tipik olarak −0.07 ile −0.3 arası. 16 hop dolaşmak (≈ −2) teslimden açıkça kötü, hemen düşmek (−1) de teslimden kötü. Böylece ajan "hemen düşür" ya da "sonsuz dolaş" davranışına kaymaz.

4.3 Eğitim

- **Double DQN:** online ağ bir sonraki action'ı seçer, target ağ değerlendirir. Target her 500 adımda kopyalanır.
- **Hiperparametreler:**
  - Replay buffer 50.000, batch 128, her 4 geçişte bir adım.
  - Adam, lr 1e-3, Huber loss, gradyan kırpma 1.0. γ = 0.98.
  - ε: 1.0 → 0.05, ilk %60 episode'da doğrusal.
- **Next-state değeri:** bir sonraki router'ın komşuları üzerinden max Q. Bu, Q-routing'in (Boyan & Littman, 1994) derin öğrenmeli hali.
- **Episode:** bir simülasyon koşusu (20 s) ve her seferinde rastgele bir senaryo:
  - agent sayısı 300–2.000
  - hız 0–20 m/s
  - yük 0.5–4
  - failure 0 / 0.1 / 0.2
  - R 2–4
- **Veri:** eğitim sadece train intent'leriyle yapılır.
- **Doğrulama ve model seçimi:**
  - Her 10 episode'da sabit 3 senaryo × 2 seed üzerinde ε = 0 ile doğrulama yapılır.
  - En iyi **doğrulama getirisine** sahip ağırlıklar saklanır.
  - Getiri seçildi çünkü RL'in optimize ettiği şey bu: teslim + gecikme birlikte. Sadece başarı oranına bakmak daha hızlı ama eşit başarılı bir policy'yi ödüllendirmez.
- **Süre (M2 CPU):** MLP 300 episode ≈ 20 dk, GNN ≈ 30 dk.

5. Deneyler ve görselleştirme (yapay zeka desteğiyle)

- **Deneyler:**
  - Her script bir parametreyi tarar: agent sayısı, hız, failure, R, yük, S-FIB özeti, header.
  - 6 yöntem × 5 seed çalıştırılır, her koşu 30 s simülasyon zamanı (~280 istek).
  - Koşular 6 paralel süreçte, en ağır koşular önce.
  - Grafiklerde ortalama ve %95 güven aralığı (gölge) var.
- **Grafik stili:**
  - Her yöntemin tek ve sabit bir rengi ve işaretçi şekli var. Renk paleti renk körlüğü için doğrulandı.
  - İnce çizgiler, yatay ince grid, lejant grafiğin üstünde.
- **Animasyon:**
  - Demo logundaki gerçek olaylardan bir senaryo kuruluyor: duyurular → bir isteğin yolu → bir handover.
  - Karşılaştırma videosu sadece logda DQN'in gerçekten daha iyi olduğu bir istek varsa üretiliyor.
- **Streamlit:** simülatörü doğrudan çağırıyor. Sonuç tablosu, Altair grafikleri (üzerine gelince değer gösteriyor), tek isteğin ağ üzerindeki yolu, hazır grafikler.
- **React + D3 oynatıcı (Vite):**
  - Sadece log dosyalarını okuyor.
  - D3 ölçekler ve zoom için, React SVG çizimi için kullanılıyor.
  - Zaman çizgisi (oynat, geri oynat, hız, kaydırıcı), olay akışı, hop başına gecikme şelalesi, başarısızlık nedenleri.

6. Karşılaşılan sorunlar ve çözümler

6.1 MiniLM eşleşmesi zayıftı

Kısa açıklamalarla top-1 doğruluk 0.61'di. Prototip vektörlerle (açıklama + örnek istekler) 0.77'ye çıktı (bkz. 2.2). Kalan hatalar birbirine benzeyen sınıflarda (örneğin görüntü sınıfları). Bu, bütün anlamsal yöntemlerin başarısına bir tavan koyuyor. Raporda bunu açıkça yazıyoruz: global bilgiye sahip registry bile bu tavanın üstüne çıkamıyor.

6.2 İlk senaryoda yönlendirmenin önemi yoktu

- **İlk ayar:** her sınıf eşit dağılımlı ve hücre başına 40 agent vardı.
- **Ölçtük:** greedy isteklerin çoğunu kendi hücresinde ya da 1 hop ötede buluyordu (ortalama 1.07 hop). Greedy'nin registry'ye karşı kaybettiği isteklerin neredeyse hepsi "yanlış agent kabul etti" nedeniyleydi, bunu hiçbir next-hop kararı değiştiremez. DQN'in öğreneceği bir şey yoktu, doğrulama başarısı greedy ile aynı kaldı.
- **Gerçekçi düzeltme:**
  - Yetenek arzını Zipf(1) yaptık: gerçekte birkaç yetenek çok yaygın, çoğu nadir.
  - Hücre yoğunluğunu 20 agent'a indirdik.
- **Sonuç:** greedy ortalama 2.4 hop'ta buluyor ve hotspot router'lardaki kuyruk kayıpları artık next-hop seçimine bağlı. DQN bundan sonra yeniden eğitildi.

6.3 Ölçeklenebilirlik

- S-FIB sayaçları numpy dizisi olarak tutuluyor, duyurular BFS ile hesaplanıyor.

- Arka plan trafiği analitik (M/M/1) hesaplanıyor.
- Flooding kopyaları router başına bir kez işleniyor.
- Sonuç: 10.000 agent / 450 router'lık bir koşu 3–25 s sürüyor.

6.4 Model seçimi

İlk eğitimde doğrulama ölçütü başarı oranıydı. DQN aynı başarıyla daha hızlı teslim etse bile bu görünmüyordu. Ölçütü doğrulama getirisi yaptık (bkz. 4.3).

7. Sonuçlar kısaca (5 seed ortalaması)

Ayrıntılar ve grafikler final raporun 5. bölümünde. Rakamların kaynağı `results/tables.md` ve `results/summary.json`.

Soru	Cevap
En hızlı yöntem hangisi?	AgentRoute (DQN/GNN): 10.000 agent'ta 21 ms. Greedy 23 ms, flooding 52 ms, registry 202 ms.
Mesaj maliyeti?	10.000 agent'ta istek başına AgentRoute 315, registry 567, flooding 1.230, Chord 4.976 mesaj
Başarı oranı?	10.000 agent'ta flooding %61.5, registry %55.7, AgentRoute %52.6, greedy %51.6, Chord %0
Neden flooding daha başarılı?	Flooding ve registry bütün adayları karşılaştırıp en iyisini seçiyor. Hop hop yöntem yolda τ'yu geçen ilk agent'a teslim ediyor ve bu agent isteklerin %14–15'inde yanlış sınıftan oluyor (flooding/registry'de %10–11).
DQN greedy'den iyi mi?	Başarıda aynı (±1 puan), 300 agent'tan itibaren %7–14 daha hızlı. Varsayılan yükte kuyruk taşmalarını neredeyse sıfırlıyor (%4.3 → %0.4), ama daha çok paketin TTL'i bitiyor (%28 → %33). En yüksek yükte (4x) greedy'nin 3.7 puan gerisinde: kayıp TTL'den değil kuyruk taşmasından geliyor (%17.6, greedy'de %12.2). Nedeni analiz edilmedi, bu açık sorun.
Mobilite?	30 m/s'de bile bütün yöntemler 1–2 puan kaybediyor. Bir handover 10.000 agent'ta 24 S-FIB mesajı tutuyor, bu tam bir duyuru turunun %0.8'i.
Node failure?	%30 çöken router'da AgentRoute %49.7 → %47.4. Registry, kendi router'ının çöktüğü seed'lerde %14–17'ye düşüyor (tek hata noktası).
R ne olmalı?	R = 2–3 başarının neredeyse hepsini veriyor, kontrol trafiği R ile artıp R = 5'te doyuyor.
S-FIB özeti?	Ortalama %48.6, k = 4 %50.6, k = 8 %52.4, tam tablo %55.6 (greedy)
Header?	int8 (388 B) bedava, binary (48 B) başarıyı yarıya indiriyor